OOP PRACTICAL FILE

MINIBANK REPORT

1. MiniBank/src/model/BankInfo.java

CODE:

```
package model;

public record BankInfo(String name, String branch) {
}
```

This code is from Practical 1.

2. MiniBank/src/model/MenuOption.java

CODE:

```
package model;

public enum MenuOption {
   OPEN_ACCOUNT,
   DEPOSIT,
   WITHDRAW,
   TRANSFER,
   STATEMENT,
   LIST_ACCOUNTS,
   REPORT,
   BATCH_DEPOSITS,
   EXIT
}
```

This code is from Practicals 1 and 14.

3. MiniBank/src/model/Customer.java

CODE:

```
package model;

import model.annotation.Id;
import java.io.Serializable;

public class Customer implements Cloneable, Serializable {

   private static final long serialVersionUID = 1L;

   private String name;
   private String email;
   private String mobile;
   @Id
   private final String customerId;
   private static long customerCounter = 100;
   private Address address;

   private static String generateCustomerId() {
      customerCounter++;
      return String.format("CUST%03d", customerCounter);
```

```
}

public Customer(String name, String email, String mobile) {
   this.customerId = generateCustomerId();
   this.name = name;
   this.email = email;
   this.mobile = mobile;
}

public String getName() {
   return name;
}

public String getEmail() {
   return email;
}

public String getMobile() {
   return mobile;
}

public String getCustomerId() {
   return customerId;
}

public Address getAddress() {
   return address;
}

public void setAddress(Address address) {
   this.address = address;
}

@Override
public Customer clone() {
   try {
      Customer copy = (Customer) super.clone();
      if (address != null) {
         copy.address = new Address(address.line, address.city, address.pincode);
      }
      return copy;
   } catch (CloneNotSupportedException e) {
      throw new AssertionError(e);
   }
}

public static class Address implements Serializable {
   private static final long serialVersionUID = 1L;
   private String line;
   private String city;
   private String pincode;

   public Address(String line, String city, String pincode) {
      this.line = line;
      this.city = city;
      this.pincode = pincode;
   }

   public String getLine() {
      return line;
   }

   public String getCity() {
      return city;
   }
```

```
        public String getPincode() {
            return pincode;
        }
    }
}
```

This code is from Practicals 2, 3, 7 and 11.

4. MiniBank/src/model/Account.java

CODE:

```
package model;

import exception.AccountNotFoundException;
import exception.BankException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

import java.io.Serializable;
import java.util.Objects;

public abstract class Account implements Transactable, InterestBearing,
        Comparable<Account>, Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @MaxLength(20)
    private final String accountNumber;

    private String ownerName;

    @Positive
    private long balance;

    private boolean active;
    private static long accountCounter;

    public static synchronized void observeAccountNumber(String number) {
        if (number != null && number.matches("AC\\d+")) {
            try {
                accountCounter = Math.max(accountCounter,
                        Long.parseLong(number.substring(2)));
            } catch (NumberFormatException ignored) {
            }
        }
    }

    private static synchronized String generateAccountNumber() {
        accountCounter++;
        return String.format("AC%04d", accountCounter);
    }

    public Account(String ownerName, long balance) {
        this(generateAccountNumber(), ownerName, balance);
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    protected Account(String accountNumber, String ownerName, long balance) {
        this.accountNumber = accountNumber;
```

```
    observeAccountNumber(accountNumber);
    this.ownerName = ownerName;
    this.balance = balance;
    this.active = true;
}

public String getAccountNumber() {
    return accountNumber;
}

public String getOwnerName() {
    return ownerName;
}

public synchronized long getBalance() {
    return balance;
}

public synchronized boolean isActive() {
    return active;
}

public synchronized void closeAccount() {
    active = false;
}

@Override
public synchronized void deposit(long amount) throws InvalidAmountException {
    if (amount <= 0) {
        throw new InvalidAmountException(
                "Deposit amount must be greater than 0");
    }
    if (!active) {
        throw new InvalidAmountException("Account is inactive");
    }
    balance += amount;
}

@Override
public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException {
    if (amount <= 0) {
        throw new InvalidAmountException(
                "Withdrawal amount must be greater than 0");
    }
    if (!active) {
        throw new InvalidAmountException("Account is inactive");
    }
    if (!canWithdraw(amount)) {
        long shortfall = amount - getAvailableForWithdrawal();
        throw new InsufficientFundsException(
                "Insufficient funds. Shortfall: " + shortfall,
                shortfall);
    }
    balance -= amount;
    return true;
}

protected long getAvailableForWithdrawal() {
    return balance;
}

public void transfer(Account to, long amount) throws BankException {
    if (to == null) {
        throw new AccountNotFoundException("Destination account not found");
```

```
    }
    if (to == this) {
        throw new InvalidAmountException(
                "Source and destination accounts must be different");
    }

    Account first = accountNumber.compareTo(to.accountNumber) < 0
            ? this : to;
    Account second = first == this ? to : this;

    synchronized (first) {
        synchronized (second) {
            long originalFromBalance = balance;
            long originalToBalance = to.balance;
            boolean completed = false;

            try {
                if (amount <= 0) {
                    throw new InvalidAmountException(
                            "Transfer amount must be greater than 0");
                }
                if (!active || !to.active) {
                    throw new InvalidAmountException(
                            "Both accounts must be active");
                }
                if (!canWithdraw(amount)) {
                    long shortfall = amount - getAvailableForWithdrawal();
                    throw new InsufficientFundsException(
                            "Insufficient funds. Shortfall: " + shortfall,
                            shortfall);
                }

                balance -= amount;
                to.balance += amount;
                completed = true;
            } catch (BankException e) {
                throw e;
            } finally {
                if (!completed) {
                    balance = originalFromBalance;
                    to.balance = originalToBalance;
                }
            }
        }
    }
}

@Override
public String toString() {
    return getAccountNumber() + " | " + getOwnerName() + " | " + getBalance();
}

@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (!(obj instanceof Account other)) {
        return false;
    }
    return accountNumber.equals(other.accountNumber);
}

@Override
public int hashCode() {
```

```
        return Objects.hash(accountNumber);
    }

    @Override
    public int compareTo(Account other) {
        return accountNumber.compareTo(other.accountNumber);
    }

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);
}
```

This code is from Practicals 2, 3, 5, 7, 8, 9, 11 and 12.

5. MiniBank/src/model/SavingsAccount.java

CODE:

```
package model;

import java.io.Serializable;
public class SavingsAccount extends Account implements Serializable {

    private static final long serialVersionUID = 1L;

    private long minBalance;

    public SavingsAccount(String ownerName, long balance, long minBalance) {
        super(ownerName, balance);
        this.minBalance = minBalance;
    }

    @Override
    public double interestRate() {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return getBalance() - amount >= minBalance;
    }

    @Override
    protected long getAvailableForWithdrawal() {
        return Math.max(0, getBalance() - minBalance);
    }
}
```

This code is from Practicals 5 and 11.

6. MiniBank/src/model/CurrentAccount.java

CODE:

```
package model;

import java.io.Serializable;
public class CurrentAccount extends Account implements Serializable {

    private static final long serialVersionUID = 1L;

    private long overdraftLimit;

    public CurrentAccount(String ownerName, long balance, long overdraftLimit) {
        super(ownerName, balance);
        this.overdraftLimit = overdraftLimit;
    }
```

```
    @Override
    public double interestRate() {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return getBalance() - amount >= -overdraftLimit;
    }

    @Override
    protected long getAvailableForWithdrawal() {
        return getBalance() + overdraftLimit;
    }
}
```

This code is from Practicals 5 and 11.

7. MiniBank/src/model/FixedDepositAccount.java

CODE:

```
package model;

import java.io.Serializable;
public class FixedDepositAccount extends Account implements Premium, Serializable {

    private static final long serialVersionUID = 1L;

    public FixedDepositAccount(String ownerName, long balance) {
        super(ownerName, balance);
    }

    @Override
    public double interestRate() {
        return 7.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return false;
    }

    @Override
    protected long getAvailableForWithdrawal() {
        return 0;
    }
}
```

This code is from Practicals 5, 6 and 11.

8. MiniBank/src/model/Transactable.java

CODE:

```
package model;

import exception.InsufficientFundsException;
import exception.InvalidAmountException;

public interface Transactable {
    void deposit(long amount) throws InvalidAmountException;
    boolean withdraw(long amount) throws InsufficientFundsException, InvalidAmountException;
}
```

This code is from Practicals 6 and 8.

9. MiniBank/src/model/InterestBearing.java

CODE:

```
package model;

public interface InterestBearing {
   double interestRate();

   long getBalance();

   default double yearlyInterest() {
      return calculateInterest(getBalance(), interestRate());
   }

   private double calculateInterest(long balance, double rate) {
      return balance * rate / 100.0;
   }

   static double calculateInterestFor(long balance, double rate) {
      return balance * rate / 100.0;
   }
}
```

This code is from Practical 6.

10. MiniBank/src/model/Premium.java

CODE:

```
package model;

public interface Premium {
}
```

This code is from Practical 6.

11. MiniBank/src/model/Command.java

CODE:

```
package model;

public record Command(TransactionType type, String accountNumber, long amount) {
}
```

This code is from Practical 4.

12. MiniBank/src/model/TransactionType.java

CODE:

```
package model;

public enum TransactionType {
   DEPOSIT,
   WITHDRAW,
   TRANSFER
}
```

This code is from Practical 4.

13. MiniBank/src/model/annotation/Id.java

CODE:

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
```

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Id {
}
```

This code is from Practical 7.

14. MiniBank/src/model/annotation/Positive.java

CODE:

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Positive {
   String message() default "must be > 0";
}
```

This code is from Practical 7.

15. MiniBank/src/model/annotation/MaxLength.java

CODE:

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface MaxLength {
   int value();
}
```

This code is from Practical 7.

16. MiniBank/src/util/Validator.java

CODE:

```
package util;

import java.util.regex.Pattern;
public class Validator {

   private static final Pattern MOBILE = Pattern.compile("^[6-9][0-9]{9}$");
   private static final Pattern EMAIL = Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
   private static final Pattern PAN = Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");
   private static final Pattern IFSC = Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

   public static boolean isValidMobile(String value) {
      return value != null && MOBILE.matcher(value).matches();
   }

   public static boolean isValidEmail(String value) {
      return value != null && EMAIL.matcher(value).matches();
   }

   public static boolean isValidPan(String value) {
      return value != null && PAN.matcher(value).matches();
   }
```

```
    public static boolean isValidIfsc(String value) {
        return value != null && IFSC.matcher(value).matches();
    }
}
```

This code is from Practical 4.

17. MiniBank/src/util/CommandParser.java

CODE:

```
package util;

import model.Command;
import model.TransactionType;

import java.util.Locale;

public class CommandParser {

    public static Command parse(String line) {
        if (line == null || line.isBlank()) {
            throw new IllegalArgumentException("Command cannot be empty");
        }

        String[] parts = line.trim().split("\\s+");
        if (parts.length != 3) {
            throw new IllegalArgumentException(
                    "Command must contain type, account number and amount");
        }

        TransactionType type = TransactionType.valueOf(
                parts[0].toUpperCase(Locale.ROOT));
        long amount = Long.parseLong(parts[2]);
        return new Command(type, parts[1], amount);
    }
}
```

This code is from Practical 4.

18. MiniBank/src/util/StatementFormatter.java

CODE:

```
package util;

import model.Account;
public class StatementFormatter {

    public static String buildStatement(Account account) {
        StringBuilder statement = new StringBuilder();
        statement.append("Account Number: ")
                .append(account.getAccountNumber()).append("\n");
        statement.append("Owner: ").append(account.getOwnerName()).append("\n");
        statement.append("Balance: ").append(account.getBalance()).append("\n");
        statement.append("Interest Rate: ")
                .append(account.interestRate()).append("%\n");
        statement.append("Yearly Interest: ")
                .append(account.yearlyInterest()).append("\n");
        statement.append("Active: ").append(account.isActive());
        return statement.toString();
    }
}
```

This code is from Practical 4.

19. MiniBank/src/util/WithdrawRule.java

CODE:

```
package util;

import model.Account;
@FunctionalInterface
public interface WithdrawRule {
   boolean allow(Account account, long amount);
}
```

This code is from Practical 6.

20. MiniBank/src/util/AnnotationValidator.java

CODE:

```
package util;

import model.annotation.MaxLength;
import model.annotation.Positive;

import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

public class AnnotationValidator {

   public static String[] validate(Object obj) {
      List<String> errors = new ArrayList<>();
      if (obj == null) {
         return new String[]{"Object cannot be null"};
      }

      Class<?> current = obj.getClass();
      while (current != null) {
         Field[] fields = current.getDeclaredFields();

         for (Field field : fields) {
            field.setAccessible(true);
            try {
               Object value = field.get(obj);

               if (field.isAnnotationPresent(Positive.class)) {
                  if (value == null || ((Number) value).doubleValue() <= 0) {
                     Positive annotation = field.getAnnotation(Positive.class);
                     errors.add(field.getName() + " " + annotation.message());
                  }
               }

               if (field.isAnnotationPresent(MaxLength.class)) {
                  MaxLength annotation = field.getAnnotation(MaxLength.class);
                  if (value != null
                        && value.toString().length() > annotation.value()) {
                     errors.add(field.getName() + " must have at most "
                           + annotation.value() + " characters");
                  }
               }
            } catch (Exception e) {
               errors.add("Error validating " + field.getName());
            }
         }
         current = current.getSuperclass();
      }
      return errors.toArray(new String[0]);
   }
```

```
}
```

This code is from Practical 7.

21. MiniBank/src/exception/BankException.java

CODE:

```
package exception;

public class BankException extends Exception {
   private static final long serialVersionUID = 1L;

   public BankException(String message) {
      super(message);
   }
}
```

This code is from Practical 8.

22. MiniBank/src/exception/AccountNotFoundException.java

CODE:

```
package exception;

public class AccountNotFoundException extends BankException {
   private static final long serialVersionUID = 1L;

   public AccountNotFoundException(String message) {
      super(message);
   }
}
```

This code is from Practical 8.

23. MiniBank/src/exception/InvalidAmountException.java

CODE:

```
package exception;

public class InvalidAmountException extends BankException {
   private static final long serialVersionUID = 1L;

   public InvalidAmountException(String message) {
      super(message);
   }
}
```

This code is from Practical 8.

24. MiniBank/src/exception/InsufficientFundsException.java

CODE:

```
package exception;

public class InsufficientFundsException extends BankException {
   private static final long serialVersionUID = 1L;

   private final long shortfall;

   public InsufficientFundsException(String message, long shortfall) {
      super(message);
      this.shortfall = shortfall;
   }

   public long getShortfall() {
      return shortfall;
   }
```

```
}
```

This code is from Practical 8.

25. MiniBank/src/util/BankResource.java

CODE:

```
package util;

public final class BankResource implements AutoCloseable {
   private boolean closed;

   public BankResource() {
      System.out.println("MiniBank resource opened.");
   }

   @Override
   public void close() {
      if (!closed) {
         closed = true;
         System.out.println("MiniBank resource closed.");
      }
   }
}
```

This code is from Practical 8.

26. MiniBank/src/service/AccountWorker.java

CODE:

```
package service;

import exception.InvalidAmountException;
import model.Account;

public class AccountWorker implements Runnable {

   private final Account account;
   private final int times;
   private final long amount;

   public AccountWorker(Account account, int times, long amount) {
      this.account = account;
      this.times = times;
      this.amount = amount;
   }

   @Override
   public void run() {
      System.out.println(Thread.currentThread().getName() + " started");
      for (int i = 0; i < times; i++) {
         try {
            account.deposit(amount);
         } catch (InvalidAmountException e) {
            System.out.println(e.getMessage());
            return;
         }
      }
      System.out.println(Thread.currentThread().getName() + " finished");
   }
}
```

This code is from Practical 9.

27. MiniBank/src/util/TransactionBuffer.java

CODE:

```
package util;

public class TransactionBuffer {
   private final Runnable[] items;
   private int head;
   private int tail;
   private int size;

   public TransactionBuffer(int capacity) {
      if (capacity < 1) {
         throw new IllegalArgumentException("Capacity must be positive");
      }
      items = new Runnable[capacity];
   }

   public synchronized void put(Runnable task) throws InterruptedException {
      while (size == items.length) {
         wait();
      }
      items[tail] = task;
      tail = (tail + 1) % items.length;
      size++;
      notifyAll();
   }

   public synchronized Runnable take() throws InterruptedException {
      while (size == 0) {
         wait();
      }
      Runnable task = items[head];
      items[head] = null;
      head = (head + 1) % items.length;
      size--;
      notifyAll();
      return task;
   }
}
```

This code is from Practical 10.

28. MiniBank/src/service/TransactionProcessor.java

CODE:

```
package service;

import model.Account;
import util.TransactionBuffer;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class TransactionProcessor implements AutoCloseable {

   private final ExecutorService executor = Executors.newFixedThreadPool(4);
   private final ConcurrentHashMap<String, Account> sharedAccounts =
         new ConcurrentHashMap<>();
   public TransactionProcessor(Iterable<Account> accounts) {
```

```
    for (Account account : accounts) {
        sharedAccounts.put(account.getAccountNumber(), account);
    }
}

public void register(Account account) {
    sharedAccounts.put(account.getAccountNumber(), account);
}

public Account get(String accountNumber) {
    return sharedAccounts.get(accountNumber);
}

public void submit(Runnable task) {
    executor.execute(task);
}

public void processBatch(List<Runnable> tasks) throws InterruptedException {
    if (tasks.isEmpty()) {
        return;
    }

    TransactionBuffer buffer = new TransactionBuffer(8);
    CountDownLatch completed = new CountDownLatch(tasks.size());

    Thread dispatcher = new Thread(() -> {
        try {
            while (true) {
                Runnable task = buffer.take();
                if (task == null) {
                    break;
                }
                executor.execute(() -> {
                    try {
                        task.run();
                    } finally {
                        completed.countDown();
                    }
                });
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "transaction-dispatcher");

    dispatcher.start();
    try {
        for (Runnable task : tasks) {
            buffer.put(task);
        }
        buffer.put(null);
        dispatcher.join();
        completed.await();
    } catch (InterruptedException e) {
        dispatcher.interrupt();
        Thread.currentThread().interrupt();
        throw e;
    }
}

public void runDeposits(String accountNumber, long amount, int count)
        throws InterruptedException {
    Account account = sharedAccounts.get(accountNumber);
    if (account == null) {
        throw new IllegalArgumentException("Account not found");
```

```
        }
        List<Runnable> tasks = new ArrayList<>();
        for (int i = 0; i < count; i++) {
            tasks.add(() -> {
                try {
                    account.deposit(amount);
                } catch (Exception e) {
                    throw new IllegalStateException(e);
                }
            });
        }
        processBatch(tasks);
    }

    public void stop() {
        executor.shutdown();
        try {
            if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
                executor.shutdownNow();
            }
        } catch (InterruptedException e) {
            executor.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }

    @Override
    public void close() {
        stop();
    }
}
```

This code is from Practicals 10, 12 and 14.

29. MiniBank/src/util/TransactionLog.java

CODE:

```
package util;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.List;

public final class TransactionLog {

    private TransactionLog() {
    }

    public static synchronized void append(Path file, String line)
            throws IOException {
        Path parent = file.toAbsolutePath().getParent();
        if (parent != null) {
            Files.createDirectories(parent);
        }
        Files.write(file, List.of(line), StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }
}
```

This code is from Practical 11.

30. MiniBank/src/util/StatePersister.java

CODE:

```
package util;

import model.Account;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.nio.file.Files;
import java.nio.file.Path;

public final class StatePersister {

    private StatePersister() {
    }

    public static void save(Account[] accounts, Path file) throws IOException {
        Path parent = file.toAbsolutePath().getParent();
        if (parent != null) {
            Files.createDirectories(parent);
        }
        try (ObjectOutputStream output = new ObjectOutputStream(
                Files.newOutputStream(file))) {
            output.writeObject(accounts);
        }
    }

    public static Account[] load(Path file)
            throws IOException, ClassNotFoundException {
        try (ObjectInputStream input = new ObjectInputStream(
                Files.newInputStream(file))) {
            Account[] accounts = (Account[]) input.readObject();
            for (Account account : accounts) {
                Account.observeAccountNumber(account.getAccountNumber());
            }
            return accounts;
        }
    }
}
```

This code is from Practical 11.

31. MiniBank/src/util/ReportGenerator.java

CODE:

```
package util;

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.attribute.BasicFileAttributes;

public final class ReportGenerator {

    private ReportGenerator() {
    }

    public static String generate(Path logsDirectory, Path reportFile)
            throws IOException {
        Files.createDirectories(logsDirectory);
        long deposits = 0;
```

```
        long withdrawals = 0;
        StringBuilder details = new StringBuilder();

        try (DirectoryStream<Path> files = Files.newDirectoryStream(
                logsDirectory, "*.log")) {
            for (Path file : files) {
                BasicFileAttributes attributes = Files.readAttributes(
                        file, BasicFileAttributes.class);
                details.append(file.getFileName())
                        .append(" | size=").append(attributes.size())
                        .append(" bytes | modified=")
                        .append(attributes.lastModifiedTime())
                        .append(System.lineSeparator());

                try (BufferedReader reader = Files.newBufferedReader(
                        file, StandardCharsets.UTF_8)) {
                    String line;
                    while ((line = reader.readLine()) != null) {
                        String[] parts = line.trim().split("\\s+");
                        if (parts.length < 3) {
                            continue;
                        }
                        try {
                            long amount = Long.parseLong(parts[parts.length - 1]);
                            if (parts[0].equalsIgnoreCase("DEPOSIT")) {
                                deposits += amount;
                            } else if (parts[0].equalsIgnoreCase("WITHDRAW")) {
                                withdrawals += amount;
                            }
                        } catch (NumberFormatException ignored) {
                        }
                    }
                }
            }
        }

        String report = "MINIBANK END-OF-DAY REPORT" + System.lineSeparator()
                + "Total deposits: " + deposits + System.lineSeparator()
                + "Total withdrawals: " + withdrawals + System.lineSeparator()
                + "Net change: " + (deposits - withdrawals)
                + System.lineSeparator() + "Files:" + System.lineSeparator()
                + details;

        Path parent = reportFile.toAbsolutePath().getParent();
        if (parent != null) {
            Files.createDirectories(parent);
        }
        Files.writeString(reportFile, report, StandardCharsets.UTF_8);
        return report;
    }
}
```

This code is from Practicals 11 and 14.

32. MiniBank/src/util/AccountStore.java

CODE:

```
package util;

import model.Account;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;

public class AccountStore {
    private final HashMap<String, Account> accounts = new HashMap<>();
```

```
    private final ArrayList<String> history = new ArrayList<>();

    public Account add(Account account) {
        if (account == null) {
            throw new IllegalArgumentException("Account is required");
        }
        accounts.put(account.getAccountNumber(), account);
        return account;
    }

    public Account get(String accountNumber) {
        return accounts.get(accountNumber);
    }

    public ArrayList<Account> getAll() {
        return new ArrayList<>(accounts.values());
    }

    public ArrayList<Account> sortedByBalance() {
        ArrayList<Account> sorted = getAll();
        sorted.sort(Comparator.comparingLong(Account::getBalance).reversed()
                .thenComparing(Account::getAccountNumber));
        return sorted;
    }

    public void record(String transaction) {
        history.add(transaction);
    }

    public ArrayList<String> getHistory() {
        return new ArrayList<>(history);
    }
}
```

This code is from Practical 12.

33. MiniBank/src/util/GenericStatistics.java

CODE:

```
package util;

public final class GenericStatistics {
    private GenericStatistics() {
    }

    public static <N extends Number> double average(N[] values) {
        if (values == null || values.length == 0) {
            throw new IllegalArgumentException("Values cannot be empty");
        }

        double total = 0;
        for (N value : values) {
            total += value.doubleValue();
        }
        return total / values.length;
    }
}
```

This code is from Practical 13.

34. MiniBank/src/util/Repository.java

CODE:

```
package util;

import model.annotation.Id;
```

```
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.HashMap;

public class Repository<T> {

   private final HashMap<String, T> items = new HashMap<>();

   public T save(T entity) {
      if (entity == null) {
         throw new IllegalArgumentException("Entity cannot be null");
      }
      items.put(readId(entity), entity);
      return entity;
   }

   @SafeVarargs
   public final void saveAll(T... values) {
      for (T value : values) {
         save(value);
      }
   }

   public T findById(String id) {
      return items.get(id);
   }

   public ArrayList<T> findAll() {
      return new ArrayList<>(items.values());
   }

   public boolean delete(String id) {
      return items.remove(id) != null;
   }

   private String readId(T entity) {
      Class<?> type = entity.getClass();
      while (type != null) {
         for (Field field : type.getDeclaredFields()) {
            if (field.isAnnotationPresent(Id.class)) {
               try {
                  field.setAccessible(true);
                  Object value = field.get(entity);
                  if (value == null) {
                     throw new IllegalArgumentException(
                           "@Id field cannot be null");
                  }
                  return value.toString();
               } catch (ReflectiveOperationException e) {
                  throw new IllegalStateException(
                        "Cannot read @Id field", e);
               }
            }
         }
         type = type.getSuperclass();
      }
      throw new IllegalArgumentException(
            "No field annotated with @Id on " + entity.getClass().getName());
   }
}
```

This code is from Practicals 13 and 14.

35. MiniBank/src/service/BankService.java

CODE:

```
package service;

import exception.AccountNotFoundException;
import exception.BankException;
import exception.InvalidAmountException;
import model.Account;
import model.CurrentAccount;
import model.Customer;
import model.FixedDepositAccount;
import model.SavingsAccount;
import util.Repository;
import util.ReportGenerator;
import util.StatePersister;
import util.StatementFormatter;
import util.TransactionLog;
import util.WithdrawRule;

import java.io.IOException;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.atomic.AtomicReference;

import static util.Validator.*;

public class BankService {

   private final Repository<Account> accounts;
   private final Repository<Customer> customers = new Repository<>();
   private final Path stateFile;
   private final Path logDirectory;
   private final Path reportFile;

   public BankService(Repository<Account> accounts, Path stateFile,
                      Path logDirectory, Path reportFile) {
      this.accounts = accounts;
      this.stateFile = stateFile;
      this.logDirectory = logDirectory;
      this.reportFile = reportFile;
   }

   public Account openAccount(String type, String owner, String email,
                              String mobile, long openingBalance)
         throws BankException {
      if (owner == null || owner.isBlank()) {
         throw new InvalidAmountException("Owner name is required");
      }
      if (!isValidEmail(email)) {
         throw new InvalidAmountException("Invalid email address");
      }
      if (!isValidMobile(mobile)) {
         throw new InvalidAmountException("Invalid mobile number");
      }
      if (openingBalance < 0) {
         throw new InvalidAmountException(
               "Opening balance cannot be negative");
      }
      if (type == null) {
         throw new InvalidAmountException("Account type is required");
      }
```

```
        Account account = switch (type.toUpperCase(Locale.ROOT)) {
            case "SAVINGS" -> new SavingsAccount(owner, openingBalance, 0);
            case "CURRENT" -> new CurrentAccount(owner, openingBalance, 1000);
            case "FIXED" -> new FixedDepositAccount(owner, openingBalance);
            default -> throw new InvalidAmountException(
                    "Account type must be SAVINGS, CURRENT or FIXED");
        };

        accounts.save(account);
        customers.save(new Customer(owner, email, mobile));
        return account;
    }

    public void deposit(String id, long amount)
            throws BankException, IOException {
        Account account = require(id);
        account.deposit(amount);
        TransactionLog.append(logDirectory.resolve("transactions.log"),
                "DEPOSIT " + id + " " + amount);
    }

    public void withdraw(String id, long amount)
            throws BankException, IOException {
        Account account = require(id);
        account.withdraw(amount);
        TransactionLog.append(logDirectory.resolve("transactions.log"),
                "WITHDRAW " + id + " " + amount);
    }

    public void transfer(String fromId, String toId, long amount)
            throws BankException, IOException {
        Account from = require(fromId);
        Account to = require(toId);
        from.transfer(to, amount);
        TransactionLog.append(logDirectory.resolve("transactions.log"),
                "WITHDRAW " + fromId + " " + amount);
        TransactionLog.append(logDirectory.resolve("transactions.log"),
                "DEPOSIT " + toId + " " + amount);
    }

    public String statement(String id) throws AccountNotFoundException {
        return StatementFormatter.buildStatement(require(id));
    }

    public List<Account> listAccounts() {
        ArrayList<Account> result = accounts.findAll();
        result.sort(Comparator.comparingLong(Account::getBalance).reversed()
                .thenComparing(Account::getAccountNumber));
        return result;
    }

    public Account getAccount(String id) {
        return accounts.findById(id);
    }

    public String generateReport() throws IOException {
        return ReportGenerator.generate(logDirectory, reportFile);
    }

    public void save() throws IOException {
        StatePersister.save(accounts.findAll().toArray(Account[]::new), stateFile);
    }

    public void processConcurrentDeposits(String id, long amount, int count)
            throws InterruptedException, BankException, IOException {
```

```
        if (count < 1 || count > 10000) {
            throw new InvalidAmountException("Batch count must be 1-10000");
        }
        if (amount <= 0) {
            throw new InvalidAmountException("Amount must be positive");
        }

        Account account = require(id);
        AtomicReference<BankException> failure = new AtomicReference<>();
        List<Runnable> tasks = new ArrayList<>();

        try (TransactionProcessor processor =
                 new TransactionProcessor(accounts.findAll())) {
            for (int i = 0; i < count; i++) {
                tasks.add(() -> {
                    Account shared = processor.get(id);
                    try {
                        if (shared == null) {
                            throw new AccountNotFoundException(
                                    "Account not found: " + id);
                        }
                        shared.deposit(amount);
                    } catch (BankException e) {
                        failure.compareAndSet(null, e);
                    }
                });
            }
            processor.processBatch(tasks);
        }

        if (failure.get() != null) {
            throw failure.get();
        }

        Path transactionFile = logDirectory.resolve("transactions.log");
        for (int i = 0; i < count; i++) {
            TransactionLog.append(transactionFile,
                    "DEPOSIT " + id + " " + amount);
        }
        System.out.println("Processed " + count + " deposits. Balance: "
                + account.getBalance());
    }

    private Account require(String id) throws AccountNotFoundException {
        Account account = accounts.findById(id);
        if (account == null) {
            throw new AccountNotFoundException("Account not found: " + id);
        }
        return account;
    }

    public boolean checkWithdrawRuleExamples(Account account, long amount) {
        WithdrawRule anonymousRule = new WithdrawRule() {
            @Override
            public boolean allow(Account target, long value) {
                return target.canWithdraw(value);
            }
        };
        WithdrawRule lambdaRule = (target, value) -> target.canWithdraw(value);
        return anonymousRule.allow(account, amount)
                == lambdaRule.allow(account, amount);
    }
}
```

This code is from Practical 14.

36. MiniBank/src/service/MiniBank.java

CODE:

```
package service;

import exception.BankException;
import exception.InvalidAmountException;
import model.Account;
import model.BankInfo;
import model.MenuOption;
import util.BankResource;
import util.Repository;
import util.StatePersister;
import util.WithdrawRule;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Scanner;

public class MiniBank {

   private static final Path DATA = Path.of("data");
   private static final Path STATE = DATA.resolve("accounts.dat");
   private static final Path LOGS = DATA.resolve("logs");
   private static final Path REPORT = DATA.resolve("report.txt");

   private static final Repository<Account> repository = new Repository<>();
   private static BankService bank;

   @SuppressWarnings("try")
   public static void main(String[] args) {
      try (BankResource resource = new BankResource()) {
         System.out.println(new BankInfo("MiniBank", "Main Branch"));
         Files.createDirectories(LOGS);
         if (Files.exists(STATE)) {
            for (Account account : StatePersister.load(STATE)) {
               repository.save(account);
            }
         }
         bank = new BankService(repository, STATE, LOGS, REPORT);
         runMenu();
      } catch (Exception e) {
         System.out.println("MiniBank error: " + e.getMessage());
      } finally {
         try {
            if (bank != null) {
               bank.save();
            }
         } catch (IOException e) {
            System.out.println("Could not save account state: " + e.getMessage());
         }
      }
   }

   private static void runMenu() throws IOException {
      try (Scanner scanner = new Scanner(System.in)) {
         boolean running = true;
         while (running) {
            printMenu();
            System.out.print("Choose: ");
            int choice;
            try {
               choice = Integer.parseInt(scanner.nextLine().trim());
            } catch (NumberFormatException e) {
```

```
                System.out.println("Enter a menu number.");
                continue;
            }

            MenuOption option = switch (choice) {
                case 1 -> MenuOption.OPEN_ACCOUNT;
                case 2 -> MenuOption.DEPOSIT;
                case 3 -> MenuOption.WITHDRAW;
                case 4 -> MenuOption.TRANSFER;
                case 5 -> MenuOption.STATEMENT;
                case 6 -> MenuOption.LIST_ACCOUNTS;
                case 7 -> MenuOption.REPORT;
                case 8 -> MenuOption.BATCH_DEPOSITS;
                case 9 -> MenuOption.EXIT;
                default -> null;
            };

            if (option == null) {
                System.out.println("Invalid option.");
                continue;
            }

            try {
                switch (option) {
                    case OPEN_ACCOUNT -> openAccount(scanner);
                    case DEPOSIT -> deposit(scanner);
                    case WITHDRAW -> withdraw(scanner);
                    case TRANSFER -> transfer(scanner);
                    case STATEMENT -> System.out.println(
                            bank.statement(readAccountNumber(scanner)));
                    case LIST_ACCOUNTS -> listAccounts();
                    case REPORT -> System.out.println(bank.generateReport());
                    case BATCH_DEPOSITS -> batchDeposits(scanner);
                    case EXIT -> {
                        bank.save();
                        running = false;
                    }
                }
            } catch (BankException | IOException e) {
                System.out.println("Operation failed: " + e.getMessage());
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.out.println("Operation interrupted.");
            }
        }
    }
    System.out.println("Account state saved. Goodbye.");
}

private static void printMenu() {
    System.out.println("\nMINIBANK - FINAL IMPLEMENTATION");
    System.out.println("1 Open account  2 Deposit  3 Withdraw  4 Transfer");
    System.out.println("5 Statement  6 List accounts  7 Report  8 Batch deposits  9 Exit");
}

private static void openAccount(Scanner scanner) throws BankException {
    System.out.print("Owner name: ");
    String owner = scanner.nextLine().trim();
    System.out.print("Email: ");
    String email = scanner.nextLine().trim();
    System.out.print("Mobile (10 digits starting 6-9): ");
    String mobile = scanner.nextLine().trim();
    System.out.print("Type (SAVINGS/CURRENT/FIXED): ");
    String type = scanner.nextLine().trim();
    System.out.print("Opening balance: ");
```

```
        long balance = readLong(scanner);

        Account account = bank.openAccount(type, owner, email, mobile, balance);
        System.out.println("Account opened: " + account);
    }

    private static void deposit(Scanner scanner)
            throws BankException, IOException {
        String id = readAccountNumber(scanner);
        long amount = readAmount(scanner);
        bank.deposit(id, amount);
        System.out.println("Deposit successful. Balance: "
                + bank.getAccount(id).getBalance());
    }

    private static void withdraw(Scanner scanner)
            throws BankException, IOException {
        String id = readAccountNumber(scanner);
        long amount = readAmount(scanner);
        Account account = bank.getAccount(id);
        if (account != null) {
            if (!bank.checkWithdrawRuleExamples(account, amount)) {
                throw new InvalidAmountException("Withdrawal rules disagree");
            }
        }
        bank.withdraw(id, amount);
        System.out.println("Withdrawal successful. Balance: "
                + bank.getAccount(id).getBalance());
    }

    private static void transfer(Scanner scanner)
            throws BankException, IOException {
        System.out.print("From account: ");
        String from = scanner.nextLine().trim();
        System.out.print("To account: ");
        String to = scanner.nextLine().trim();
        long amount = readAmount(scanner);
        bank.transfer(from, to, amount);
        System.out.println("Transfer successful.");
    }

    private static void listAccounts() {
        var accounts = bank.listAccounts();
        if (accounts.isEmpty()) {
            System.out.println("No accounts found.");
            return;
        }
        accounts.forEach(System.out::println);
    }

    private static void batchDeposits(Scanner scanner)
            throws BankException, IOException, InterruptedException {
        String id = readAccountNumber(scanner);
        long amount = readAmount(scanner);
        System.out.print("Number of deposits (1-10000): ");
        int count = Math.toIntExact(readLong(scanner));
        bank.processConcurrentDeposits(id, amount, count);
    }

    private static String readAccountNumber(Scanner scanner) {
        System.out.print("Account number: ");
        return scanner.nextLine().trim();
    }

    private static long readAmount(Scanner scanner) {
```

```
        System.out.print("Amount: ");
        return readLong(scanner);
    }

    private static long readLong(Scanner scanner) {
        while (true) {
            try {
                return Long.parseLong(scanner.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.print("Invalid number. Enter a whole number: ");
            }
        }
    }
}
```

This code is from Practicals 1 and 14.

37. MiniBank/test/service/BankServiceTest.java

CODE:

```
package service;

import exception.InsufficientFundsException;
import model.Account;
import model.CurrentAccount;
import model.SavingsAccount;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;
import util.Repository;
import util.StatePersister;

import java.nio.file.Path;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class BankServiceTest {

    @TempDir
    Path temp;

    private BankService service(Repository<Account> repository) {
        return new BankService(repository,
                temp.resolve("accounts.dat"),
                temp.resolve("logs"),
                temp.resolve("report.txt"));
    }

    @Test
    void depositIncreasesBalance() throws Exception {
        Repository<Account> repository = new Repository<>();
        Account account = new SavingsAccount("Riya", 1000, 0);
        repository.save(account);

        service(repository).deposit(account.getAccountNumber(), 500);

        assertEquals(1500, account.getBalance());
    }

    @Test
    void withdrawalOverSavingsLimitThrows() throws Exception {
        Repository<Account> repository = new Repository<>();
        Account account = new SavingsAccount("Riya", 1500, 1000);
        repository.save(account);
        BankService bank = service(repository);
        assertThrows(InsufficientFundsException.class,
```

```
            () -> bank.withdraw(account.getAccountNumber(), 1000));
        assertEquals(1500, account.getBalance());
    }

    @Test
    void concurrentDepositsKeepCorrectBalance() throws Exception {
        Repository<Account> repository = new Repository<>();
        Account account = new SavingsAccount("Riya", 0, 0);
        repository.save(account);

        service(repository).processConcurrentDeposits(
            account.getAccountNumber(), 1, 1000);

        assertEquals(1000, account.getBalance());
    }

    @Test
    void saveAndLoadPreservesAccountState() throws Exception {
        Account account = new SavingsAccount("Riya", 2500, 500);
        Path file = temp.resolve("accounts.dat");
        StatePersister.save(new Account[]{account}, file);

        Account[] loaded = StatePersister.load(file);

        assertEquals(1, loaded.length);
        assertEquals(account.getAccountNumber(), loaded[0].getAccountNumber());
        assertEquals(2500, loaded[0].getBalance());
    }

    @Test
    void oppositeDirectionTransfersRemainConsistent() throws Exception {
        Account first = new CurrentAccount("First", 100_000, 1000);
        Account second = new CurrentAccount("Second", 100_000, 1000);

        Thread one = new Thread(() -> transferMany(first, second));
        Thread two = new Thread(() -> transferMany(second, first));
        one.start();
        two.start();
        one.join();
        two.join();

        assertEquals(200_000, first.getBalance() + second.getBalance());
        assertEquals(100_000, first.getBalance());
        assertEquals(100_000, second.getBalance());
    }

    private static void transferMany(Account from, Account to) {
        for (int i = 0; i < 1000; i++) {
            try {
                from.transfer(to, 1);
            } catch (Exception e) {
                throw new AssertionError(e);
            }
        }
    }
}
```

This code is from Practical 14.